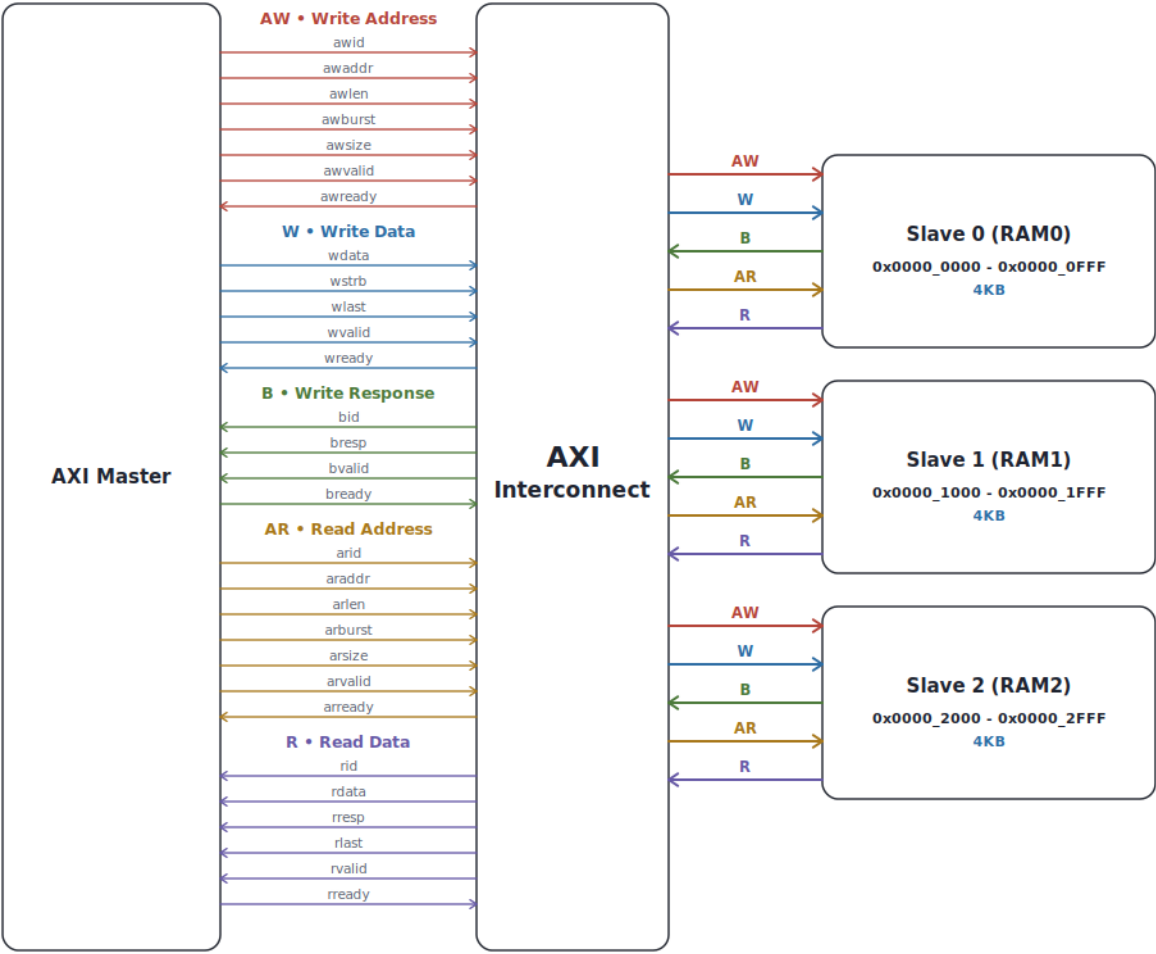


AXI 4.0 Design and Implementation



Created by

Ahmed Alaa Mohamed

Contents

Introduction	3
AMBA Overview	3
Evolution of AMPA	3
AXI Protocol Overview	4
Channel Transfers and Transactions	5
Global Signals	6
Write Address Channel (AW)	6
Write Data Channel (W)	7
Write Response Channel (B)	7
0.0.1 Read Address Channel (AR)	8
Read Data Channel (R)	8
Main AXI Features	9
Burst-Based Transactions	9
Separate Address and Data Phases	9
Multiple Outstanding Transactions	9
Out-of-Order Transaction Completion	10
Write Bursts and Byte Strobes	10
Summary of Features	11
Architecture	12
Top-Level Block Diagram	12
Manager	13
Interconnect	13
Subordinates	14
Module Hierarchy	14
Implementation	16
Master-Side Channels	16
Write Address Channel (axi_master_aw_ch)	16
Write Data Channel (axi_master_w_ch)	17
Read Data Channel (axi_master_r_ch)	19
Slave-Side Channels	19
Write Address Channel (axi_slave_aw_ch)	19
Read Data Channel (axi_slave_r_ch_ram)	21
Interconnect and Arbitration	22
Address Decoding	22
Burst Address Generation	22
WRAP Address Calculation	23

Single Burst	24
2-Beat WRAP Burst	24
4-Beat WRAP Burst	25
8-Beat INCR Burst	26
8-Beat WRAP Burst	27
16-Beat INCR Burst	28
16-Beat WRAP Burst	29
1-Byte Sub-Word Writes	30
2-Byte Sub-Word Writes	30
Interconnect Arbitration Across Independent Slaves	31

Introduction

AMBA Overview

The Advanced Microcontroller Bus Architecture (AMBA) is an open-standard, on-chip interconnect specification developed by Arm for connecting and managing functional blocks in System-on-Chip (SoC) designs. AMBA protocols define how these functional blocks communicate with one another, and are widely used across ASIC and SoC designs – from IoT subsystems to smartphones and networking chips.

AMBA offers several key benefits:

- **Efficient IP reuse** – a common interface standard reduces SoC development cost and time.
- **Flexibility** – a range of AMBA specifications are optimized for different power, performance, and area requirements.
- **Compatibility** – IP components from different vendors can be integrated seamlessly.
- **Industry support** – AMBA is widely adopted, with strong third-party IP and tool support.

Bus interface standards such as AMBA are primarily differentiated by two performance characteristics:

- **Bandwidth** – the rate at which data can be transferred across the interface. In a synchronous system, maximum bandwidth is bounded by the product of clock speed and data bus width.
- **Latency** – the delay between the initiation and completion of a transaction. In burst-based transfers, latency usually refers to the completion of the first transfer rather than the whole burst.

An efficient interface is one that approaches maximum bandwidth with minimal latency.

Evolution of AMPA

AMBA was introduced by Arm in the late 1990s and has evolved continuously to meet the demands of new processors and technologies, as shown in [Figure 1](#).

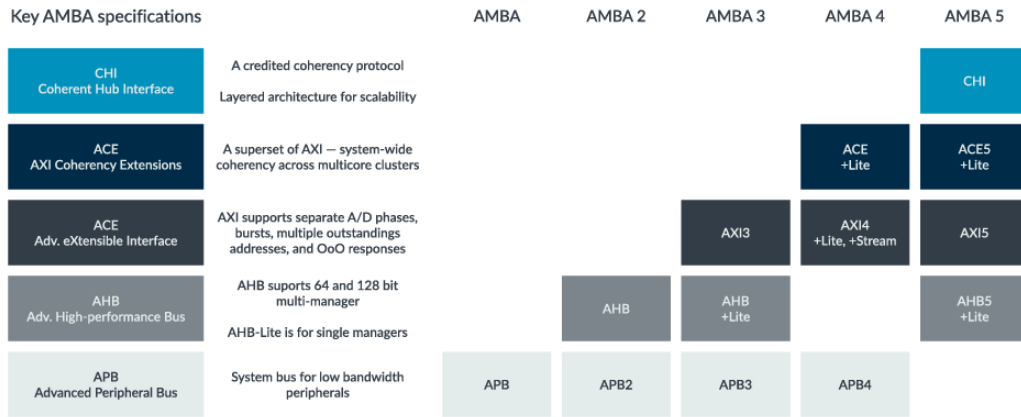


Figure 1: Evolution of key AMBA specifications across generations.

- **AMBA 1** introduced the Advanced System Bus (ASB) and the Advanced Peripheral Bus (APB). ASB has since been superseded, while APB remains widely used for simple, low-bandwidth control accesses such as peripheral register interfaces.
- **AMBA 2 (1999)** introduced the Advanced High-performance Bus (AHB), a single clock-edge, pipelined protocol. Access to the target device is arbitrated through a multiplexer, allowing one manager access at a time.
- **AMBA 3 (2003)** introduced the Advanced Trace Bus (ATB), used in CoreSight debug and trace solutions, and AHB-Lite, a simplified single-manager subset of AHB. This generation also introduced the Advanced eXtensible Interface (AXI), targeted at high-performance, high-clock-frequency designs.
- **AMBA 4 (2010)** introduced AXI4, followed by AXI Coherency Extensions (ACE) in 2011, which add system-wide coherency signaling so multiple processors can share memory. This generation also introduced AXI4-Stream, a unidirectional, manager-to-subordinate protocol with reduced signal routing suited to FPGA implementations.
- **AMBA 5 (2014 onward)** introduced the Coherent Hub Interface (CHI), featuring a redesigned, high-speed transport layer and congestion-reduction mechanisms, with multiple revisions adding new features over time. In 2016, AHB-Lite was updated to AHB5 to support the Armv8-M architecture and extend TrustZone security to the system level. In 2019, Adaptive Traffic Profiles (ATP) were introduced to model high-level memory access behavior concisely and portably. AXI5, ACE5, and ACE5-Lite further extend earlier generations with performance and scalability features aligned with CHI.

AXI Protocol Overview

AXI is an interface specification that defines the interface of IP blocks, rather than the interconnect fabric itself. There are only two AXI interface types – **manager** and **subordinate** – and these are symmetrical: every AXI connection joins a manager interface

to a subordinate interface. Because interconnect interfaces share the same signal set, integrating IP from different sources is straightforward.

A direct manager-to-subordinate connection provides maximum bandwidth with no additional logic, and since AXI defines a single protocol, only one interface needs to be validated. In a typical SoC, an Arm processor is a common example of a manager, while a memory controller is a typical subordinate.

When multiple managers and subordinates must communicate, an interconnect fabric is required. This fabric itself implements manager and subordinate interfaces, and can additionally perform protocol conversion – for example, allowing AXI3 and AXI4 components to coexist within the same SoC, which is common practice in real designs.

Channel Transfers and Transactions

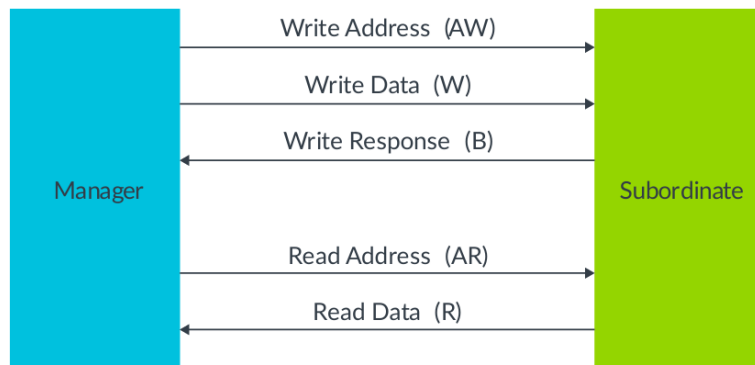


Figure 2: The five AXI channels between manager and subordinate.

The AXI protocol defines a point-to-point connection between a manager and a subordinate using five independent channels, as shown in Figure 2.

Write operations use three channels:

- **Write Address (AW)** – the manager sends the target address.
- **Write Data (W)** – the manager transfers the data to be written.
- **Write Response (B)** – the subordinate confirms completion of the write.

Read operations use two channels:

- **Read Address (AR)** – the manager sends the address to be read.
- **Read Data (R)** – the subordinate returns the requested data.

An error response may be signaled if, for example, the address is invalid, the data is corrupted, or the access lacks the required security permission.

Separating the address and data channels for reads and writes helps maximize interface bandwidth. Since there is no timing relationship between the read and write channel groups, read and write sequences can occur concurrently.

Each channel carries several signals, all sharing a common prefix that identifies the channel:

- **AW** – Write Address channel signals
- **W** – Write Data channel signals
- **B** – Write Response channel signals
- **AR** – Read Address channel signals
- **R** – Read Data channel signals

Global Signals

Signal	Source	Description
ACLK	Clock source	Global clock signal. All signals are sampled on the rising edge of the global clock.
ARESETn	Reset source	Global reset signal. This signal is active LOW.

Write Address Channel (AW)

Signal	Source	Description
AWADDR[31:0]	Manager	Write address. Gives the address of the first transfer in a write burst transaction. The associated control signals determine the addresses of the remaining transfers in the burst.
AWLEN[3:0]	Manager	Burst length. Gives the exact number of transfers in a burst, determining the number of data transfers associated with the address.
AWSIZE[2:0]	Manager	Burst size. Indicates the size of each transfer in the burst. Byte lane strobes indicate exactly which byte lanes to update.
AWBURST[1:0]	Manager	Burst type. Coupled with the size information, details how the address for each transfer within the burst is calculated.
AWVALID	Manager	Write address valid. Indicates that valid write address and control information are available: 1 = available, 0 = not available. Remains stable until AWREADY goes HIGH.
AWREADY	Subordinate	Write address ready. Indicates that the subordinate is ready to accept an address and associated control signals: 1 = ready, 0 = not ready.

Write Data Channel (W)

Signal	Source	Description
WDATA[31:0]	Manager	Write data. The write data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide.
WSTRB[3:0]	Manager	Write strobes. Indicates which byte lanes to update in memory; one write strobe per eight bits of the write data bus, so WSTRB[n] corresponds to WDATA[(8×n)+7:(8×n)].
WLAST	Manager	Write last. Indicates the last transfer in a write burst.
WVALID	Manager	Write valid. Indicates that valid write data and strobes are available: 1 = available, 0 = not available.
WREADY	Subordinate	Write ready. Indicates that the subordinate can accept the write data: 1 = ready, 0 = not ready.

Write Response Channel (B)

Signal	Source	Description
BRESP[1:0]	Subordinate	Write response. Indicates the status of the write transaction. Allowable responses are OKAY, EXOKAY, SLVERR, and DECERR.
BVALID	Subordinate	Write response valid. Indicates that a valid write response is available: 1 = available, 0 = not available.
BREADY	Manager	Response ready. Indicates that the manager can accept the response information: 1 = ready, 0 = not ready.

0.0.1 Read Address Channel (AR)

Signal	Source	Description
ARADDR[31:0]	Manager	Read address. Gives the initial address of a read burst transaction. Only the start address is provided; control signals detail how the address is calculated for the remaining transfers.
ARLEN[3:0]	Manager	Burst length. Gives the exact number of transfers in a burst, determining the number of data transfers associated with the address.
ARSIZE[2:0]	Manager	Burst size. Indicates the size of each transfer in the burst.
ARBURST[1:0]	Manager	Burst type. Coupled with the size information, details how the address for each transfer within the burst is calculated.
ARVALID	Manager	Read address valid. Indicates, when HIGH, that the read address and control information is valid and will remain stable until ARREADY is high: 1 = valid, 0 = not valid.
ARREADY	Subordinate	Read address ready. Indicates that the subordinate is ready to accept an address and associated control signals: 1 = ready, 0 = not ready.

Read Data Channel (R)

Signal	Source	Description
RDATA[31:0]	Subordinate	Read data. The read data bus can be 8, 16, 32, 64, 128, 256, 512, or 1024 bits wide.
RRESP[1:0]	Subordinate	Read response. Indicates the status of the read transfer. Allowable responses are OKAY, EXOKAY, SLVERR, and DECERR.
RLAST	Subordinate	Read last. Indicates the last transfer in a read burst.
RVALID	Subordinate	Read valid. Indicates that the required read data is available and the read transfer can complete: 1 = available, 0 = not available.
RREADY	Manager	Read ready. Indicates that the manager can accept the read data and response information: 1 = ready, 0 = not ready.

Main AXI Features

AXI was designed from the ground up for high-performance, high-frequency system designs, and its signal set reflects a small number of core architectural features that distinguish it from earlier AMBA protocols like AHB. These features are what allow AXI to sustain high throughput while remaining simple enough for straightforward IP integration.

Burst-Based Transactions

Every AXI transaction is burst-based. Rather than issuing a separate address for every data transfer, the manager issues a single address at the start of the burst, together with control information – **LEN** (burst length), **SIZE** (size of each transfer), and **BURST** (burst type) – on the address channel. The subordinate uses this control information to internally calculate the address of every subsequent transfer in the burst, so the address bus does not need to be re-driven for each beat of data. This significantly reduces address-phase overhead compared to a non-burst protocol.

Figure 3 shows a single read burst of four data beats, **D(A0)** through **D(A3)**, all associated with a single address phase (**ARADDR** = **A**). Only one address is issued on the **AR** channel, yet four data beats are transferred on the **R** channel, with **RLAST** asserted on the final beat to mark the end of the burst.

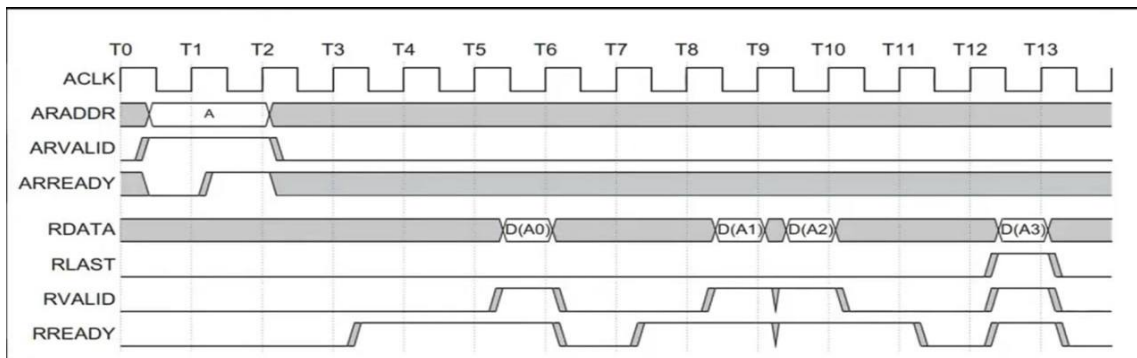


Figure 3: A single AXI read burst transaction: one address phase produces four data beats, terminated by **RLAST**.

Separate Address and Data Phases

The address/control phase and the data phase are carried on entirely separate channels (**AR/AW** for address, **R/W** for data). Because these channels are independent, the subordinate does not need to wait for the data phase of one transaction to complete before accepting the address of the next. This decoupling is what allows AXI to pipeline transactions and sustain high bandwidth, since address and data can be in flight simultaneously.

Multiple Outstanding Transactions

A manager is not required to wait for one transaction to complete before issuing the next. AXI allows multiple addresses to be outstanding at the same time, which keeps the interconnect and subordinate busy instead of stalling on round-trip latency.

Figure 4 illustrates this with two overlapping read bursts. The manager issues address A, and before the data for A has finished returning, it issues address B on the very next cycle that **ARREADY** is high. The subordinate then services both bursts, returning **D(A0)**, **D(A1)**, **D(A2)**, followed immediately by **D(B0)**, **D(B1)**, with **RLAST** marking the end of each individual burst. This pipelining of address issue ahead of data completion is a key reason AXI achieves much higher effective bandwidth than a strictly sequential protocol.

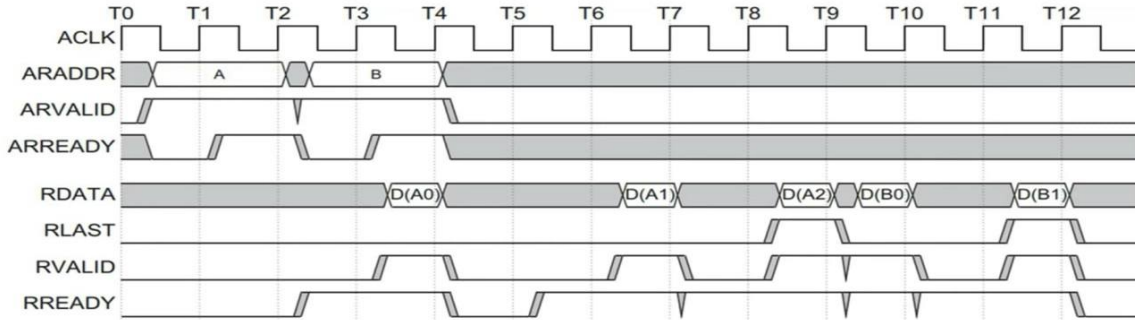


Figure 4: Overlapping read bursts: address B is issued before the data phase of burst A has completed.

Out-of-Order Transaction Completion

Because multiple transactions can be outstanding at once, AXI also permits data to be returned out of order relative to the order the addresses were issued. Each transaction can optionally carry an ID tag, and the subordinate (or interconnect) uses this tag to associate returned data with the correct original request, even if responses do not complete in the order they were requested. This is particularly useful in multi-subordinate systems, where one subordinate may respond faster than another for a transaction issued earlier.

Write Bursts and Byte Strobes

Write transactions follow the same burst principle as reads, but require an explicit handshake in both directions: the manager drives address and data, while the subordinate must accept the data (**WREADY**) before ultimately confirming the whole transaction with a single write response (**BRESP**) on the B channel, after the last data beat.

Figure 5 shows a complete write burst: a single address phase (**AWADDR** = A) is followed by four write data beats, **D(A0)** through **D(A3)**, with **WLAST** marking the final beat. Only after the burst completes does the subordinate assert **BVALID** with a **BRESP** of **OKAY**, confirming the entire burst was written successfully. Note that, unlike the read data channel, only one write response is issued per burst, not per beat.

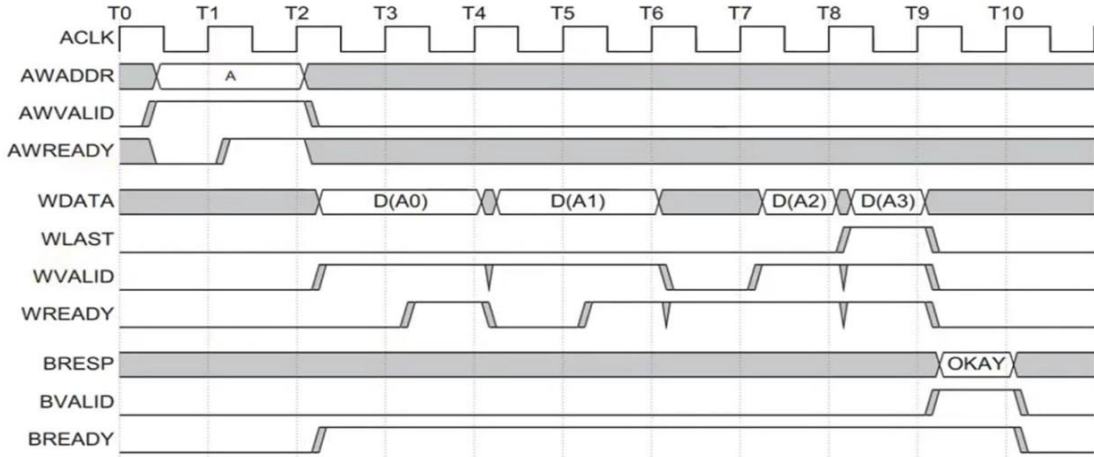


Figure 5: A complete AXI write burst: address phase, four data beats terminated by WLAST, and a single BRESP for the whole burst.

A key feature visible on the W channel is the **write strobe** signal, WSTRB. Because the write data bus can be wide (up to 1024 bits), a manager does not always want to update every byte lane on every beat – for example, when writing a single byte or a 16-bit value into a 32-bit-wide bus. WSTRB is a per-byte-lane bitmask: bit n of WSTRB corresponds to byte lane $WDATA[(8 \times n) + 7 : (8 \times n)]$, and only the byte lanes whose strobe bit is HIGH are actually written into memory by the subordinate. This allows AXI to support unaligned and sub-word writes without requiring the manager to perform a read-modify-write itself, and without wasting bandwidth transferring bytes that should not change.

Summary of Features

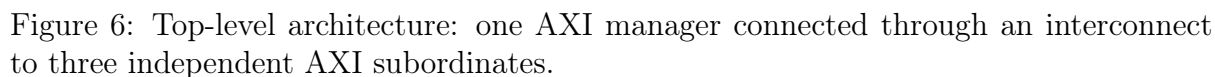
Taken together, these features are what make AXI suitable for high-performance interconnect:

- Burst-based transfers reduce address-phase overhead.
- Separate address and data channels allow pipelining.
- Multiple outstanding addresses keep the pipeline full and hide latency.
- Out-of-order completion (via transaction IDs) prevents a slow subordinate from blocking a faster one.
- Byte-level write strobes allow precise, sub-word writes without extra logic in the manager.

Architecture

Top-Level Block Diagram

Figure 6 shows the top-level architecture of the design. Unlike a simple point-to-point connection, this version introduces an **AXI_Interconnect** block sitting between a single **AXI_master_Top** and three independent subordinate instances, **AXI_Slave_Top** (RAM 1, RAM 2, RAM 3). This models a more realistic AXI system, where one manager must be able to address multiple memory-mapped subordinates over a shared set of channels.



AXI_master_Top is unchanged from the single-subordinate version. It issues write and read transactions using AXI_master_AW_channel, AXI_master_W_channel, and AXI_master_B_channel on the write side, and AXI_master_AR_channel with AXI_master_R_channel on the read side. AXI_burst_address_gen produces the sequence of addresses for a burst, and AXI_wstrb_gen produces the corresponding write strobes for each beat. The manager is completely unaware that there is more than one subordinate on the other end of the bus – from its point of view, it issues a single address and burst, and the interconnect is responsible for delivering it to the correct destination.

The `AXI_Interconnect` block sits between the manager and the three subordinates and is responsible for routing each transaction to the correct RAM instance based on its

address. It contains two independent decoders:

- **Read_addr_decoder** – inspects the incoming read address (**ARADDR**) and determines which of the three subordinates the read transaction is targeting.
- **Write_addr_decoder** – inspects the incoming write address (**AWADDR**) and determines which subordinate the write transaction is targeting.

Each decoder effectively implements an address map: the overall address space is partitioned into three regions, one per RAM instance, and the decoder steers the **AW/AR** channel handshake – and subsequently the associated **W/R** data – toward the subordinate that owns that region. Keeping the read and write decoders separate reflects the fact that AXI read and write channels are fully independent; a read transaction to RAM 2 can be in progress at the same time as a write transaction to RAM 1, since they are decoded and routed independently.

Subordinates

Each subordinate instance, **AXI_Slave_Top**, is identical in structure and repeated three times (RAM 1, RAM 2, RAM 3). Each instance contains its own **AXI_slave_AW_channel**, **AXI_slave_W_channel**, **AXI_slave_B_channel**, **AXI_slave_AR_channel**, and **AXI_slave_R_channel**, along with a **write_addr_generator** and **read_addr_generator** that convert the incoming AXI burst address into sequential internal addresses for its own local RAM. Because each subordinate is a self-contained, independent instance, RAM 1, RAM 2, and RAM 3 can service transactions completely in parallel – for example, a burst write to RAM 1 and a burst read from RAM 3 can be handled at the same time, since neither RAM instance depends on the others' internal state.

As in the single-subordinate design, each RAM block is modeled with a fixed **3-cycle read/write latency** to mimic the timing behavior of a real hardware memory, rather than responding within a single cycle. This forces each subordinate's channel logic to correctly hold its **VALID/READY** handshakes while the memory access is in flight, which is essential now that the interconnect and address decoders must also correctly track which subordinate a given outstanding transaction belongs to.

Module Hierarchy

- **AXI_master_Top**
 - **AXI_master_AW_channel** – drives the write address channel handshake.
 - **AXI_master_W_channel** – drives the write data channel handshake and interfaces with the write **AXI_fifo**.
 - **AXI_master_B_channel** – accepts the write response.
 - **AXI_master_AR_channel** – drives the read address channel handshake.
 - **AXI_master_R_channel** – accepts read data and interfaces with the read **AXI_fifo**.
 - **AXI_burst_address_gen** – generates the sequence of transfer addresses within a burst.

- `AXI_wstrb_gen` – generates the per-beat write strobe pattern.
- `AXI_fifo` ($\times 2$) – buffers write data going out and read data coming in, decoupling the burst datapath from the rest of the manager logic.
- **AXI_Interconnect**
 - `Read_addr_decoder` – routes read transactions to the correct subordinate based on `ARADDR`.
 - `Write_addr_decoder` – routes write transactions to the correct subordinate based on `AWADDR`.
- **AXI_Slave_Top** ($\times 3$, instantiated as RAM 1, RAM 2, RAM 3)
 - `AXI_slave_AW_channel` – accepts the write address handshake.
 - `AXI_slave_W_channel` – accepts write data and strobes.
 - `AXI_slave_B_channel` – issues the write response after the burst completes.
 - `AXI_slave_AR_channel` – accepts the read address handshake.
 - `AXI_slave_R_channel` – returns read data.
 - `write_addr_generator` / `read_addr_generator` – convert AXI burst addressing into sequential internal RAM addresses.
 - `RAM` – the backing memory for this instance, modeled with a 3-cycle access latency.

This structure keeps the manager, interconnect, and each subordinate fully decoupled and independently verifiable: the manager only ever talks AXI to the interconnect, the interconnect only ever routes based on address, and each subordinate only ever sees the transactions addressed to it.

Implementation

Master-Side Channels

Write Address Channel (`axi_master_aw_ch`)

The AW channel module drives write-address requests onto the AXI bus using the standard `AWVALID`/`AWREADY` handshake. It combines a small request FIFO with a three-state FSM.

Request FIFO. A `DEPTH`-entry circular FIFO buffers incoming requests (address, length, burst, size, ID) on `start_tx`, allowing multiple requests to be queued without waiting on the AXI handshake. `aw_fifo_full` signals when it's full.

Channel FSM. Figure 7 shows the FSM that drives `AWVALID`.

- **IDLE** – waits for the FIFO to be non-empty, then latches the next request and moves to **SEND**.
- **SEND** – drives `AWVALID` with the latched request until `AWREADY` completes the handshake.
- **DONE** – one-cycle state that signals completion before returning to **IDLE**.

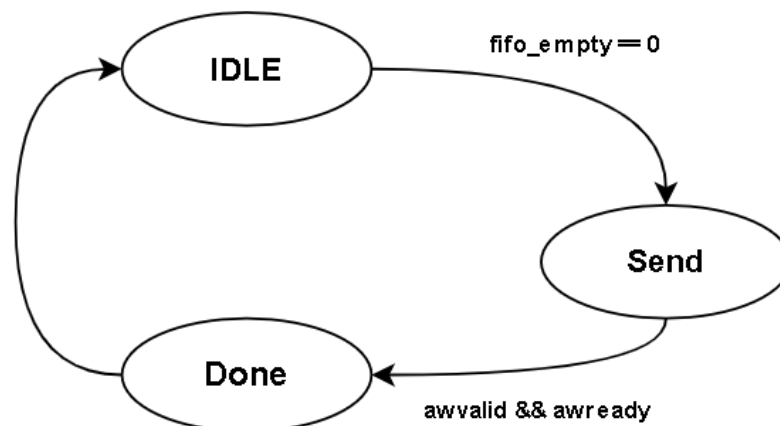


Figure 7: State machine driving the AW channel handshake.

The FIFO's read pointer only advances when the FSM leaves IDLE, keeping the FIFO and FSM in sync. The module also forwards the latched burst parameters and an `awvalid_ready` pulse to the W channel and burst-address generator.

Write Data Channel (`axi_master_w_ch`)

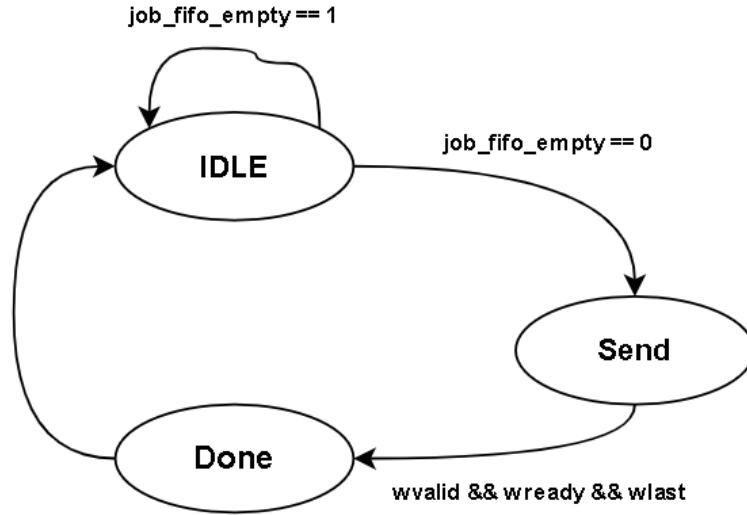


Figure 8: State machine driving the W channel handshake.

The W channel module streams write data out on the AXI bus using the `WVALID/WREADY` handshake, pulling each beat from a write-data FIFO. A separate small job FIFO captures the burst length, burst type, and ID from the AW channel each time a request is accepted (`awvalid_ready`), so the W channel knows how many beats to send and when to assert `WLAST`.

Figure 8 shows the driving FSM.

- **IDLE** – waits for the job FIFO to be non-empty, then latches the burst length/type/ID and moves to **SEND**.
- **SEND** – drives `WVALID` while data is available, presenting FIFO data on `WDATA` with its strobe, and asserts `WLAST` once the beat counter reaches the latched burst length. Each accepted beat increments the beat counter and pops the write-data FIFO.
- **DONE** – reached once `WVALID && WREADY && WLAST`, then returns to **IDLE**.

On completion, the module reports `wlast_done` and `wlast_done_id` to the B channel, signaling that a burst has finished and identifying which transaction it belongs to.

Read Address Channel (axi_master_ar_ch)

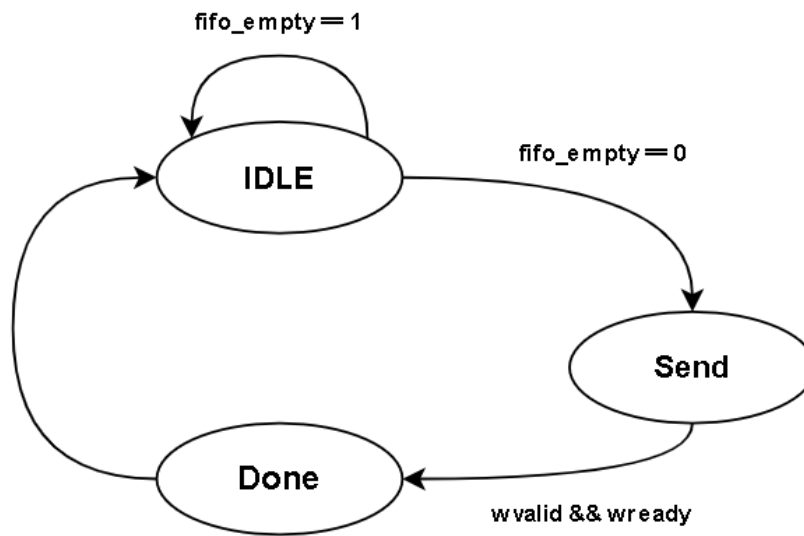


Figure 9: State machine driving the AR channel handshake.

The AR channel module mirrors the AW channel: a small request FIFO buffers incoming read requests (address, length, burst, size, ID) on `start_tx`, and the same three-state FSM (Figure 9) drives `ARVALID`.

- **IDLE** – waits for the FIFO to be non-empty, then latches the next request and moves to **SEND**.
- **SEND** – drives `ARVALID` with the latched request until `ARREADY` completes the handshake.
- **DONE** – one-cycle state that pulses `arvalid_ready` before returning to **IDLE**.

As with the AW channel, the FIFO's read pointer only advances when the FSM leaves **IDLE**, and `ar_fifo_full` provides back-pressure to the caller. The completed request's ID and burst parameters are passed downstream to the R channel via `arvalid_ready`.

Read Data Channel (axi_master_r_ch)

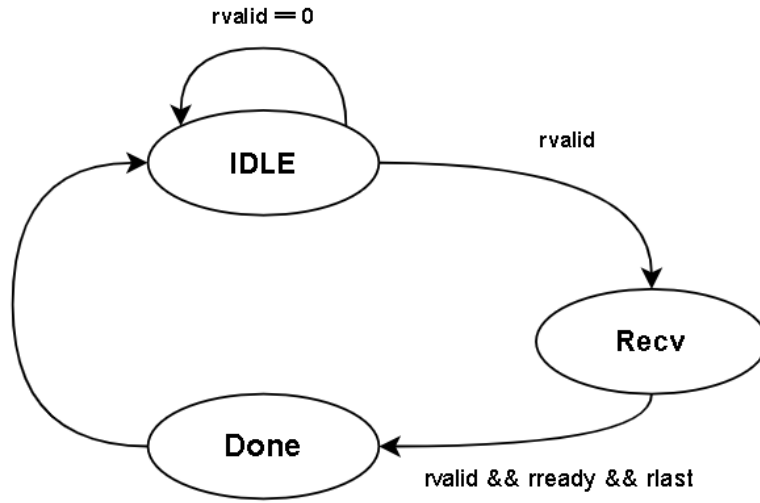


Figure 10: State machine driving the R channel handshake.

The R channel module accepts read data returned by the subordinate using the **RVALID**/**RREADY** handshake and pushes each beat into a read-data FIFO. Its FSM (Figure 10) is triggered directly by the AR channel's **arvalid_ready** rather than by a request FIFO of its own.

- **IDLE** – waits for **RVALID** to go high, then moves to **RECV**.
- **RECV** – drives **RREADY** as long as the read-data FIFO has room (**!fifo_full**), pushing each accepted beat into the FIFO. Stays in **RECV** until **RVALID && RREADY && RLAST**.
- **DONE** – one-cycle state that pulses **rx_done** and latches the transaction's ID into **rx_done_id**, before returning to **IDLE**.

Back-pressure is applied naturally: **RREADY** only asserts while the FIFO can still accept data, so the subordinate is stalled if the FIFO fills up. The completed transaction's ID is reported to the rest of the design via **rx_done**/**rx_done_id**, allowing it to be matched against the original request.

Slave-Side Channels

Write Address Channel (axi_slave_aw_ch)

The slave AW channel accepts incoming write-address requests using the **AWVALID**/**AWREADY** handshake and buffers them in a **DEPTH**-entry FIFO. **AWREADY** is driven directly from FIFO occupancy (**!fifo_full**), so the subordinate simply stops accepting new requests once the FIFO is full, without needing a separate FSM.

The head-of-queue entry (**awaddr_out**, **awlen_out**, **awburst_out**, **awsize_out**, **awid_out**) is exposed continuously to the write-address generator, and **awvalid_ready** pulses high

whenever a request is accepted. The FIFO only advances (**pop**) when downstream logic pulses **aw_pop**, so the request stays available until the address generator has consumed it.

Write Data Channel (**axi_slave_w_ch**)

The slave W channel accepts write data beats from the manager using the **WVALID**/**WREADY** handshake and forwards each accepted beat directly into the target subordinate's memory. It tracks pending write jobs (burst length and ID) in a small job queue that is pushed whenever the AW channel completes a handshake (**awvalid_ready**), so the W channel always knows the length and ID of the burst it is currently receiving.

Figure 11 shows the driving FSM, which has two states.

- **IDLE** – waits for the job queue to be non-empty (**job_empty_fifo == 0**), then moves to **RCV** to begin accepting the queued burst's data beats.
- **RCV** – drives **WREADY** as long as the memory is not full, accepting each beat (**WVALID && WREADY**) into memory. On the final beat of the burst (**WVALID && WREADY && WLAST**), the completed job is popped from the queue; if another job is already queued, the FSM stays in **RCV** to begin the next burst immediately, otherwise it returns to **IDLE**.

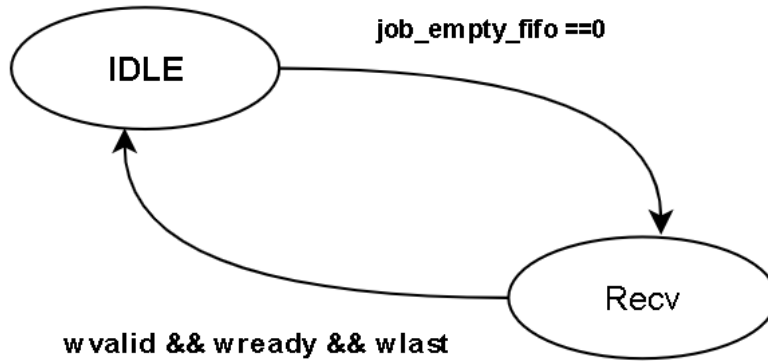


Figure 11: State machine driving the slave W channel handshake.

On completion of each burst, the module reports **wlast_done** and **wlast_done_id**, identifying which queued transaction's data phase has finished – mirroring the master-side **wlast_done**/**wlast_done_id** signals used to correlate completion with the B channel.

Read Address Channel (**axi_slave_ar_ch**)

The slave AR channel accepts incoming read-address requests using the **ARVALID**/**ARREADY** handshake and buffers them in a **DEPTH**-entry FIFO (address, length, burst, size, ID), mirroring the structure of the slave AW channel. **ARREADY** is driven directly from FIFO

occupancy, so the subordinate simply stops accepting new read requests once the FIFO is full.

Unlike the write-side channels, this module has no separate control FSM: the queue is advanced by an explicit `ar_pop` pulse from the R channel, which consumes the current head of the queue once it has been serviced. To avoid presenting invalid data on the very first request, the output address and control signals are combinationally muxed – when the FIFO is empty, they are driven directly from the incoming request on the cycle it is accepted; otherwise, they are driven from the queue head (`rd_ptr`). This ensures `araddr_out` and the associated burst controls are always valid the same cycle a request becomes available, whether it arrives into an empty queue or is read back out of it.

Read Data Channel (`axi_slave_r_ch_ram`)

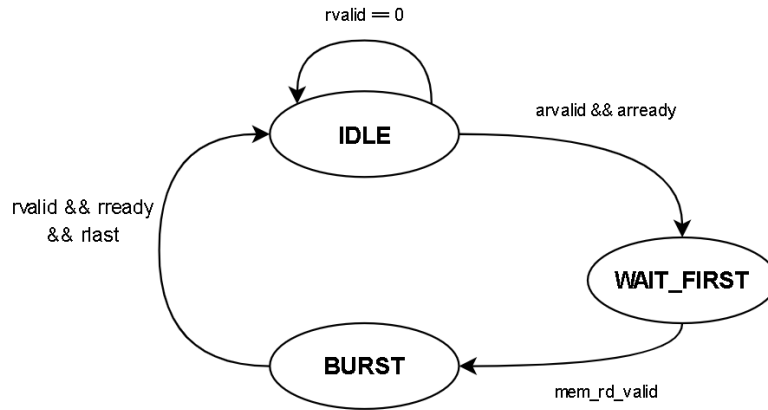


Figure 12: State machine driving the slave R channel handshake.

The slave R channel returns read data to the manager over `RVALID/RREADY`, sourcing each beat from the RAM’s read port. Since the RAM has a fixed access latency, the module pipelines requests ahead of responses using two counters: `req_cnt`, tracking beats requested from RAM (`mem_rd_en`), and `resp_cnt`, tracking beats returned to the manager. `RLAST` asserts once `resp_cnt` reaches the latched burst length.

On address acceptance (`arvalid_ready`), the module latches ID and burst length and pops the AR queue entry. The FSM has three states:

- **IDLE** – waits for `arvalid_ready`, pops the AR entry, moves to `WAIT_FIRST`.
- **WAIT_FIRST** – waits for the first `mem_rd_valid`, then moves to `BURST`.
- **BURST** – drives `RVALID` on each valid RAM read while issuing requests for subsequent beats, returning to `IDLE` once the final beat is accepted (`RVALID && RREADY && RLAST`).

`RID` is held constant from the latched ID for the full burst, and a delayed copy of `ar_pop`, `start_beat_gen`, triggers the beat-counting logic for the new transfer.

Interconnect and Arbitration

Address Decoding

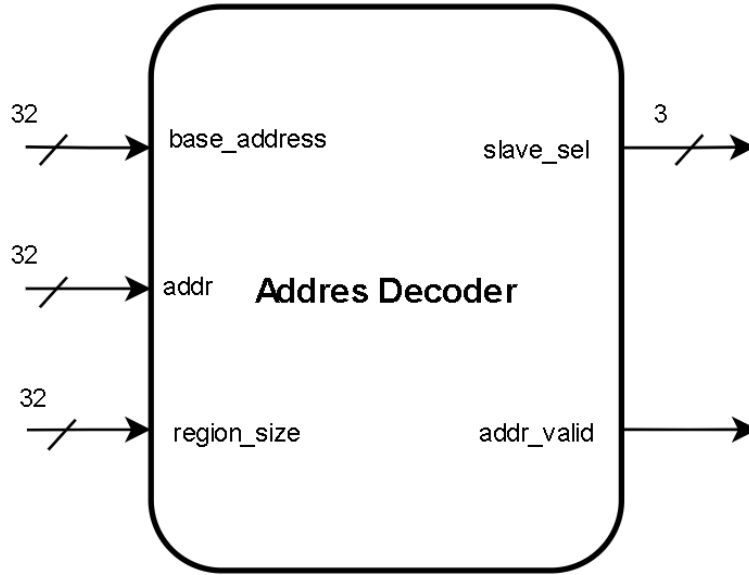


Figure 13: Interface block diagram of the address decoder.

The address decoder maps an incoming AXI address (**AWADDR** or **ARADDR**) to the subordinate that owns it. Each subordinate is assigned a configurable memory-mapped region, defined by a **base_address** and **region_size** pair, so the decoder is reusable for any number of subordinates without hardcoding address ranges.

Decoding is purely combinational: the incoming **addr** is compared against each region in turn, and the region whose base and size bound the address determines **slave_sel**. If the address does not fall within any configured region, **addr_valid** is deasserted, signaling that the transaction should be rejected with a **DECERR** response rather than routed to a subordinate.

Since the same decoder logic is used for both channels, one instance is placed on the write side (decoding **AWADDR** into **Write_addr_decoder**) and a second, identical instance on the read side (decoding **ARADDR** into **Read_addr_decoder**), consistent with the fully independent read and write paths described in Chapter 3.

Burst Address Generation

The burst address generator (**axi_burst_addr_gen**) computes the address of each beat in a burst, given a starting address (**addr_in**), the encoded transfer size (**AxSIZE**), and the burst type (**AxBURST**). On **start**, it latches the transaction; on each **beat_cnt_en** pulse it advances the address according to the latched burst type.

Three burst types are supported:

- **FIXED** – the address holds constant across all beats, used for accesses to a single register such as a FIFO port.
- **INCR** – the address increments by the transfer size (`size_bytes`) after every beat.
- **WRAP** – the address increments like INCR, but wraps back to a lower boundary once it reaches the end of an aligned address window, allowing critical-word-first cache-line accesses.

WRAP Address Calculation

For a WRAP burst, the address must wrap within a window whose size equals the total burst size (`size_bytes × number of beats`), aligned to that same size. This window is computed once, when the transaction starts, and reused for every beat:

```
wrap_boundary = size_bytes * (len_in + 1)
wrap_base     = addr_in & ~(wrap_boundary - 1)
```

on each beat:

```
if ((addr + size_bytes) % wrap_boundary == 0) and (addr != 0):
    addr <= wrap_base
else:
    addr <= addr + size_bytes
```

Because `wrap_boundary` is always a power of two, aligning `addr_in` down to `wrap_base` can be done with a simple bitmask (`addr_in & ~(wrap_boundary - 1)`) rather than a division. The address then increments beat-by-beat as in INCR mode, until it reaches the top of the aligned window, at which point it snaps back to `wrap_base` rather than continuing to climb – keeping every beat of the burst within the same naturally aligned block.

Simulation Results

Single Burst

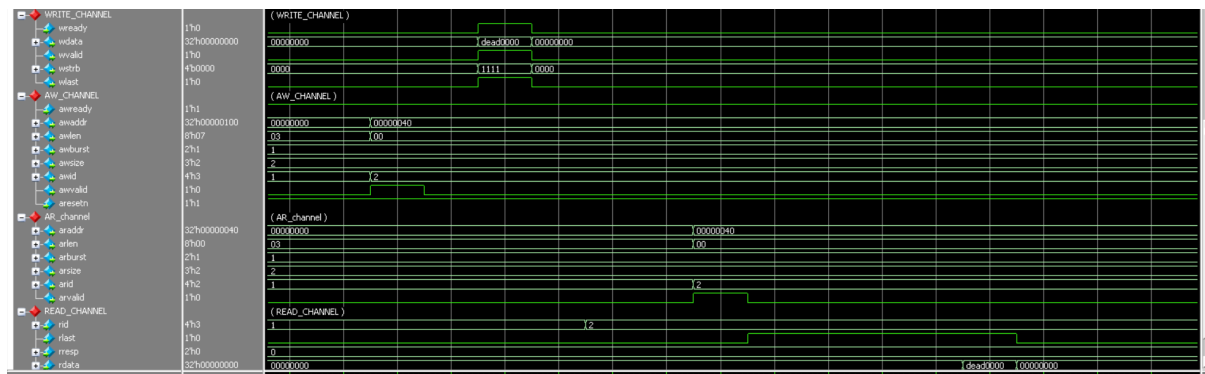


Figure 14: Waveform of a single burst

2-Beat WRAP Burst

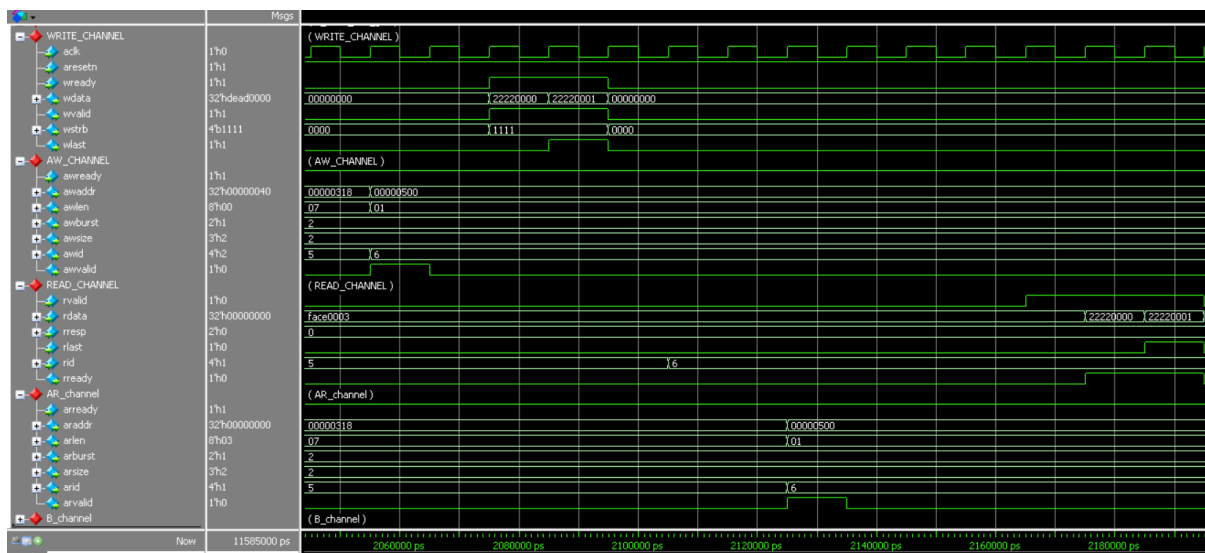


Figure 15: Waveform of a 2-beat WRAP Burst burst

4-Beat WRAP Burst

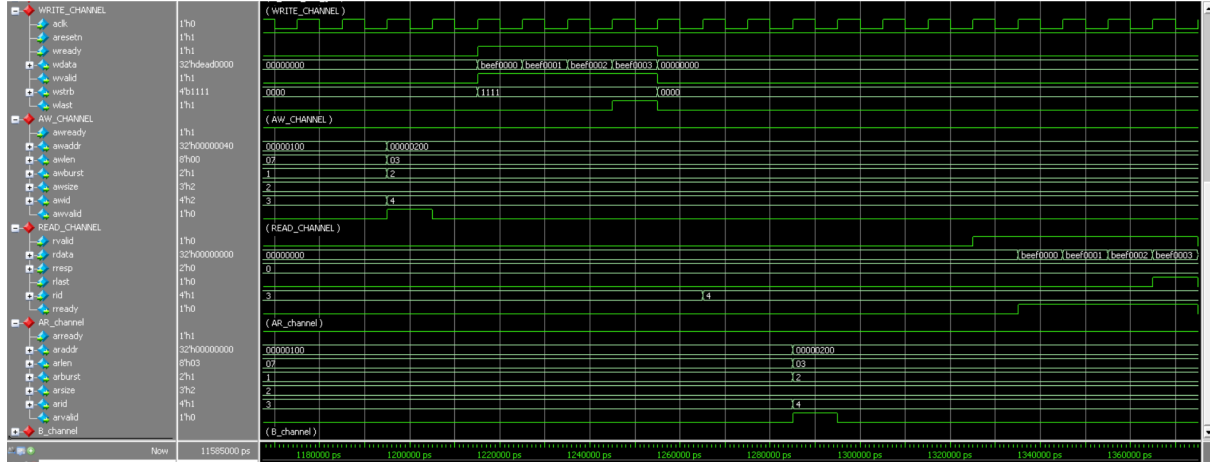


Figure 16: Waveform of a 4-beat WRAP burst write followed by a matching read, ID = 4.

A 4-beat WRAP burst ($AWLEN/ARLEN = 3$, $AWSIZE = 2$) is written to address $0x00000200$ with ID 4, then read back. The write streams `beef0000`–`beef0003` with `WLAST` correctly marking the final beat, and the read returns the identical sequence with matching `RID`, confirming correct WRAP address generation and data integrity.

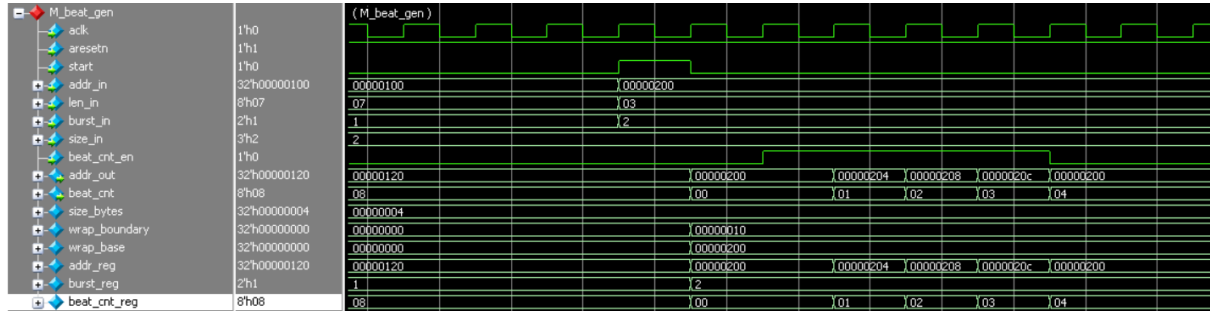


Figure 17: Internal address generation for the 4-beat WRAP burst at 0x200.

Internal signals of `axi_burst_addr_gen` for the same transaction: `wrap_boundary = 0x10` and `wrap_base = 0x200` are latched on `start`, and `addr_out` steps `0x200` → `0x204` → `0x208` → `0x200` before wrapping back to `0x200`, confirming correct WRAP boundary calculation and beat-by-beat address increment.

8-Beat INCR Burst

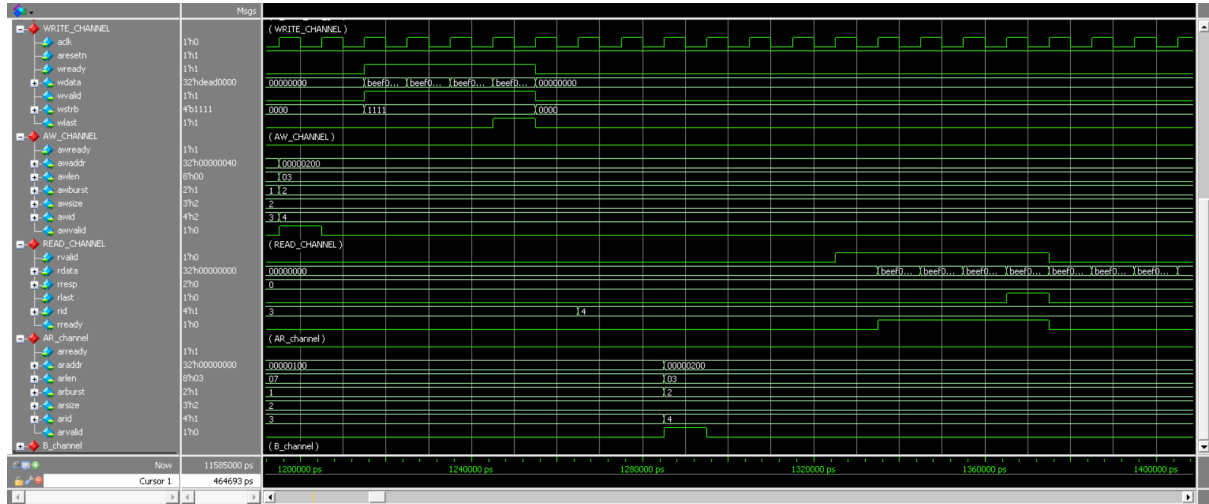


Figure 18: Waveform of an 8-beat INCR burst write followed by a matching read, ID = 4.

An 8-beat INCR burst ($AWLEN/ARLEN = 7$, $AWSIZE = 2$) is written to address $0x00000200$ with ID 4, then read back. All eight beats stream correctly with $WLAST/RLAST$ marking the final beat, and the read data matches the written sequence with matching RID.

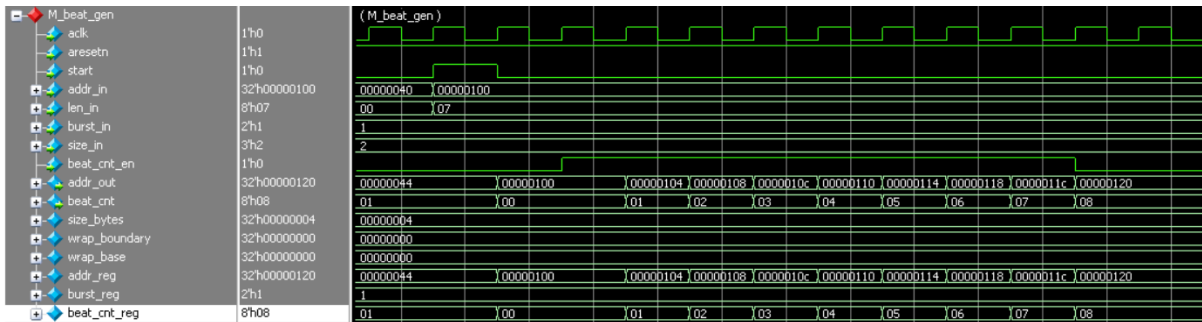


Figure 19: Internal address generation for the 8-beat INCR burst at 0x100.

Internal signals of `axi_burst_addr_gen` show `addr_out` incrementing by 4 bytes each beat – $0x100$ through $0x120$ – with `wrap_boundary/wrap_base` held at zero since INCR bursts do not wrap, confirming correct address increment behavior.

8-Beat WRAP Burst

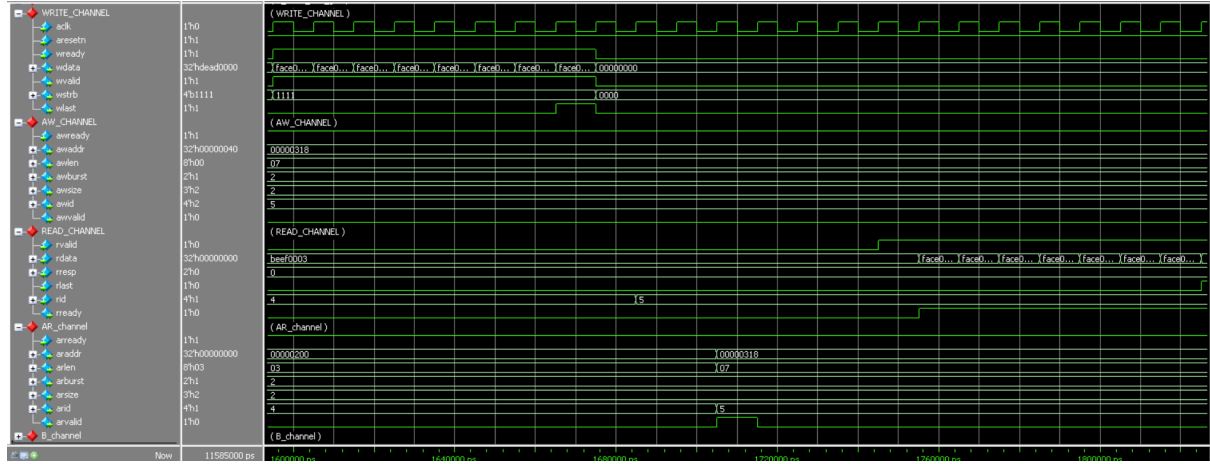


Figure 20: Waveform of an 8-beat WRAP burst write followed by a matching read, ID = 5.

An 8-beat WRAP burst ($AWLEN/ARLEN = 7$, $AWSIZE = 2$) is written to address 0x00000318 with ID 5, then read back. All eight beats stream correctly with WLAST/RLAST marking the final beat, and the read data matches the written sequence with matching RID.

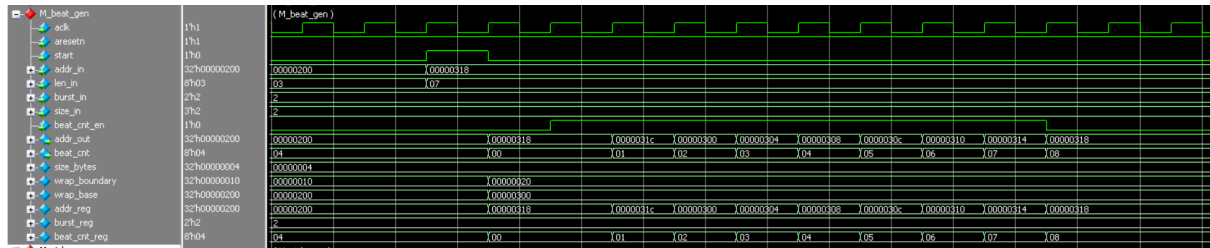


Figure 21: Internal address generation for the 8-beat WRAP burst at 0x318.

Internal signals of `axi_burst_addr_gen` show a 32-byte wrap window (`wrap_boundary = 0x20`, `wrap_base = 0x300`). `addr_out` increments from 0x318 to 0x31c, then wraps mid-burst back to 0x300 and continues incrementing (0x304 through 0x314) before returning to 0x318, confirming correct WRAP wraparound behavior when the starting address is not aligned to the window base.

16-Beat INCR Burst

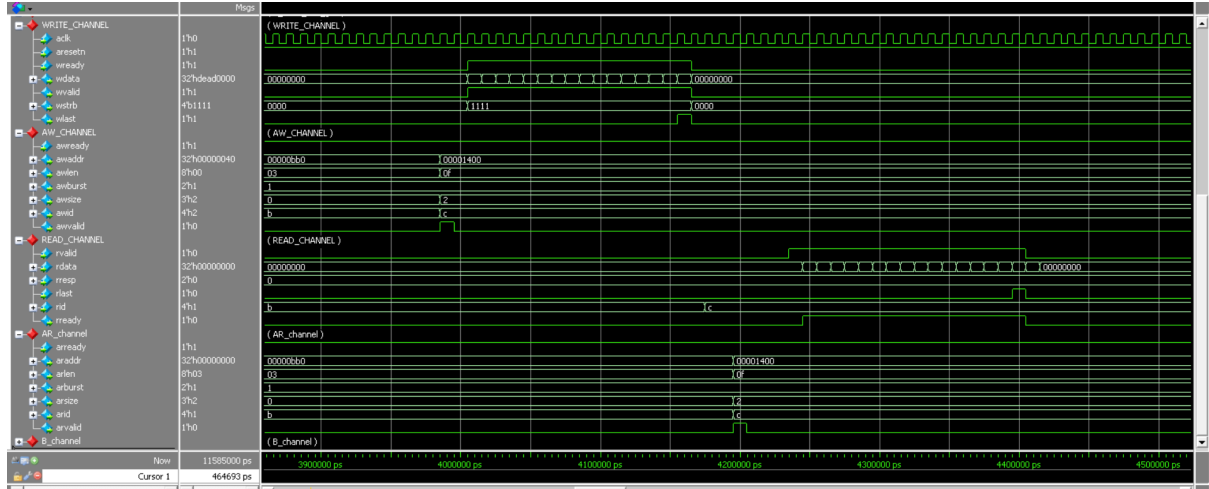


Figure 22: Waveform of a 16-beat INCR burst write followed by a matching read, ID = c.

A 16-beat INCR burst (AWLEN/ARLEN = 0xf, AWSIZE = 2) is written to address 0x00001400 with ID c, then read back. All sixteen beats stream correctly with WLAST/RLAST marking the final beat, and the read data matches the written sequence with matching RID.

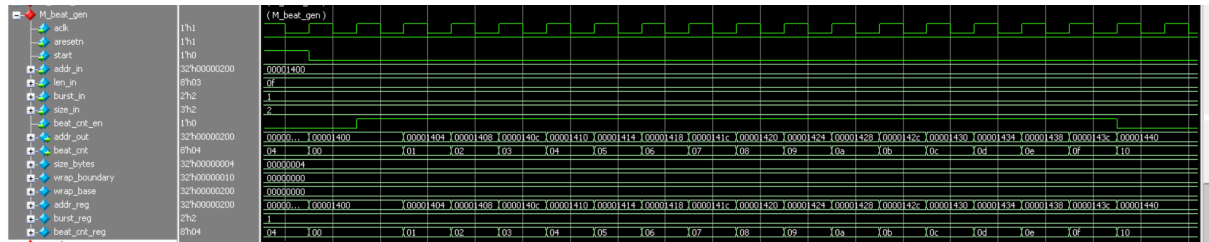


Figure 23: Internal address generation for the 16-beat INCR burst at 0x1400.

Internal signals of axi_burst_addr_gen show addr_out incrementing by 4 bytes each beat, stepping from 0x1400 through 0x1440 across all sixteen beats, confirming correct INCR address generation for a longer burst.

16-Beat WRAP Burst

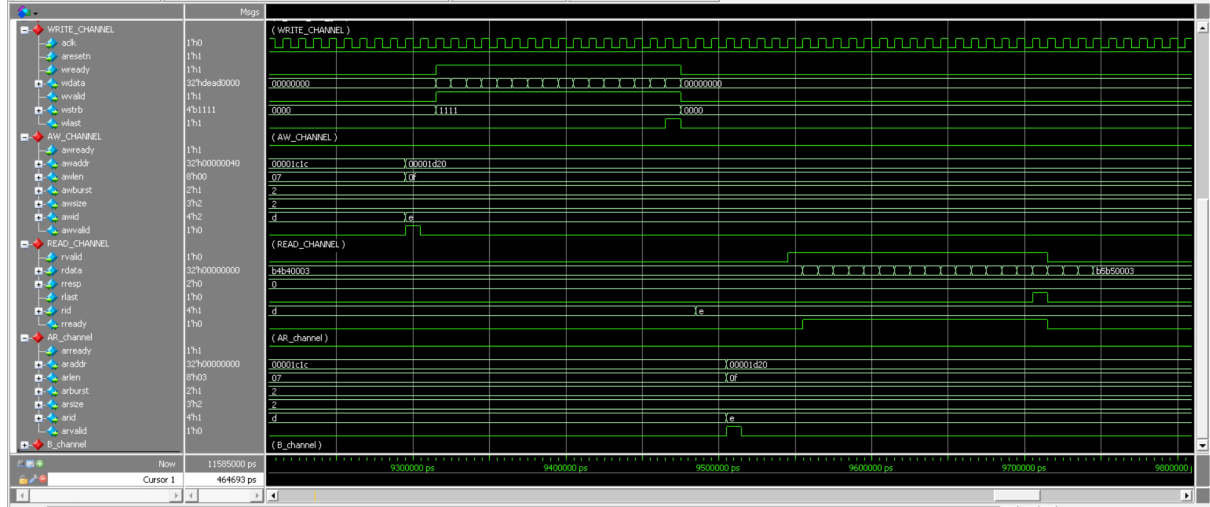


Figure 24: Waveform of a 16-beat WRAP burst write followed by a matching read, ID = e.

A 16-beat WRAP burst ($AWLEN/ARLEN = 0xf$, $AWSIZE = 2$) is written to address $0x00001d20$ with ID e, then read back. All sixteen beats stream correctly with $WLAST/RLAST$ marking the final beat, and the read data matches the written sequence with matching RID.

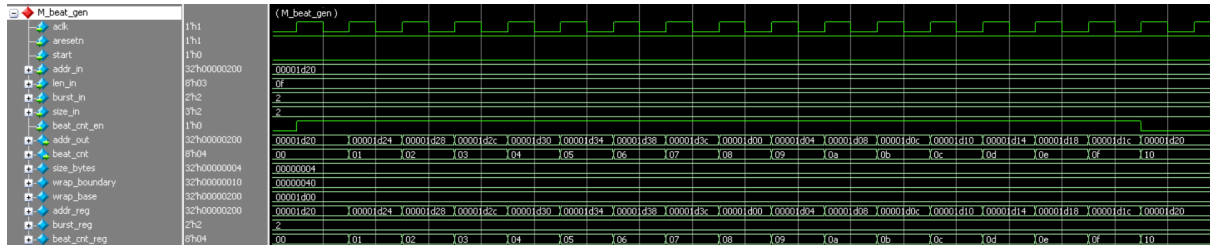


Figure 25: Internal address generation for the 16-beat WRAP burst at $0x1d20$.

Internal signals of `axi_burst_addr_gen` show a 64-byte wrap window (`wrap_boundary = 0x40`, `wrap_base = 0x1d00`). `addr_out` increments from $0x1d20$ up to $0x1d3c$, wraps back to $0x1d00$, continues incrementing through $0x1d1c$, and returns to $0x1d20$, confirming correct WRAP wraparound behavior for a larger burst with a mid-window starting address.

1-Byte Sub-Word Writes

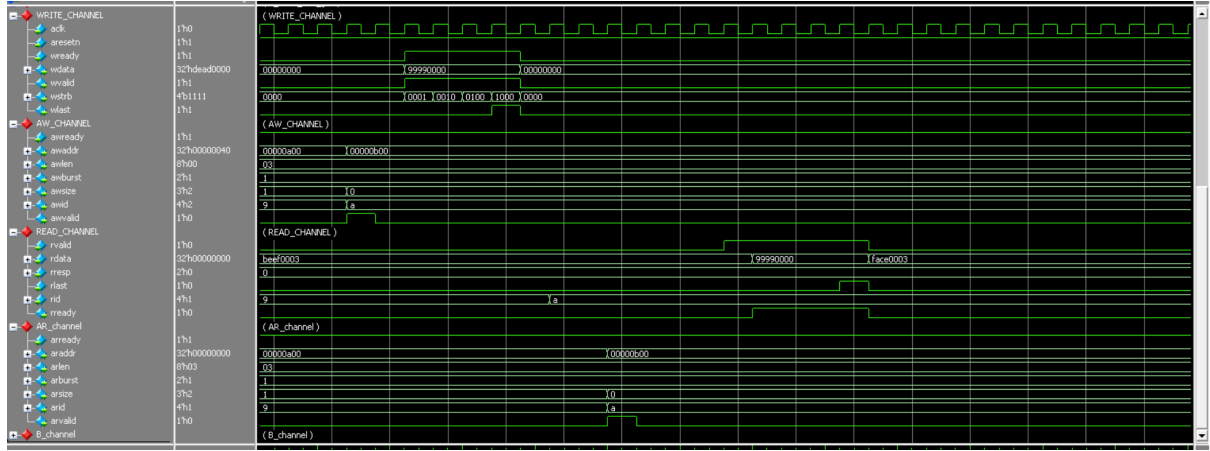


Figure 26: Internal signals of the master B channel across two back-to-back write bursts.

Internal signals of `axi_master_b_ch` show `wlast_done` pulsing for each completed burst, incrementing `outstanding_cnt` (push), and `bvalid/bready` completing the handshake with `tx_done`, which decrements the count (pop). The counter correctly tracks outstanding write responses across two consecutive bursts without overlap errors.

2-Byte Sub-Word Writes

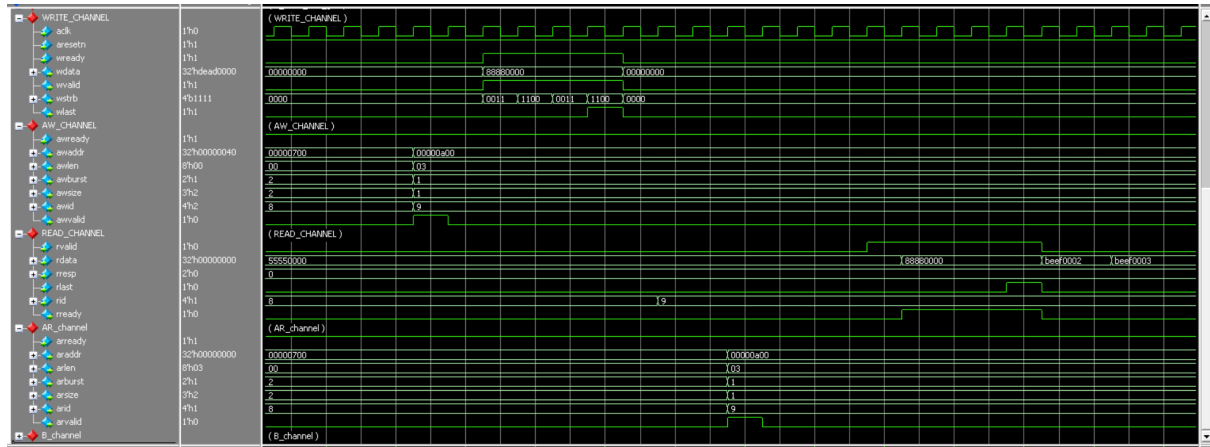


Figure 27: Waveform of writes using `AWSIZE = 1` (2 bytes per transfer) with byte-strobe masking.

Interconnect Arbitration Across Independent Slaves

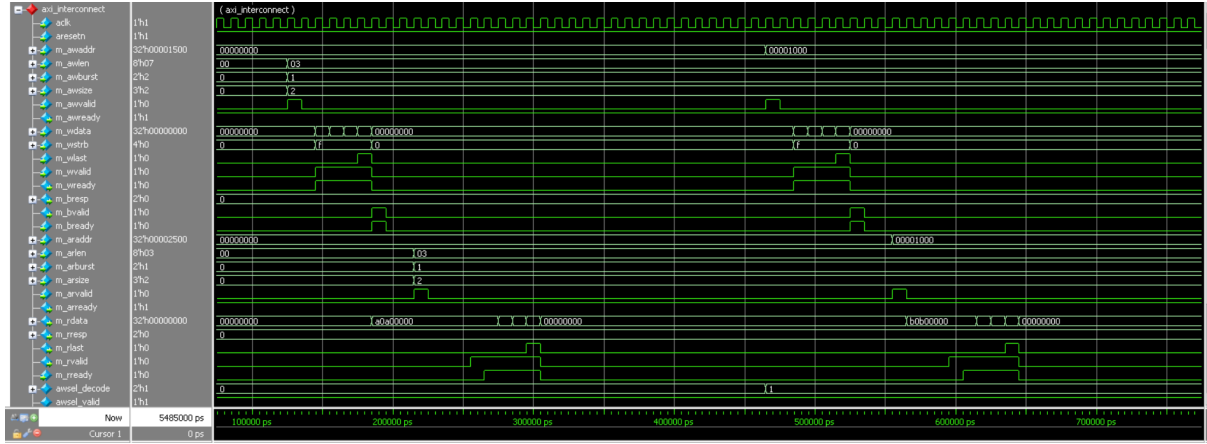


Figure 28: Internal signals of `axi_interconnect` showing two separate write/read transactions routed to different slave regions.

Two independent write/read pairs are shown: the first at address 0x00001500/0x00002500, the second at 0x00001000. `awsel_decode` updates correctly between the two, confirming the interconnect routes each transaction to the correct slave without interference.

A single-beat write to address 0x700 (ID 8) is followed by a 4-beat INCR burst to address 0xa00 (ID 9) with `AWSIZE` = 1, i.e. 2 bytes per transfer on a 32-bit bus. `WSTRB` alternates between 0011 and 1100 across beats, correctly masking only the active 2-byte lane on each transfer, and the corresponding read-back confirms the sub-word data was written to the correct byte positions.